

Documento de Requisitos Software

Servicio Web de Gestión de Gastos

1. Introducción

1.1 Propósito del sistema

El objetivo de este proyecto es desarrollar un servicio web en Java que permita a los usuarios gestionar sus gastos de forma organizada. El sistema deberá permitir registrar ingresos y gastos, clasificarlos por categorías, consultar estadísticas, gestionar presupuestos y, en una futura versión, conectarse con una aplicación móvil.

Este documento define los requisitos funcionales, no funcionales, entidades principales, casos de uso y reglas de negocio necesarias para comenzar el diseño UML y el desarrollo del sistema.

2. Alcance del sistema

El sistema permitirá a los usuarios:

- Crear una cuenta e iniciar sesión.
- Registrar gastos e ingresos.
- Clasificar movimientos por categorías.
- Consultar el historial de movimientos.
- Filtrar gastos por fecha, categoría o cantidad.
- Crear presupuestos mensuales.
- Consultar estadísticas básicas.
- Gestionar grupos de gastos compartidos.
- Preparar una API REST que pueda ser consumida por una futura aplicación móvil.

En una primera versión, el sistema puede funcionar como backend con API REST. Más adelante se podrá añadir una interfaz web o aplicación móvil.

3. Tipos de usuarios

3.1 Usuario no registrado

Usuario que todavía no tiene cuenta en el sistema.

Puede:

- Registrarse.
- Iniciar sesión si ya tiene cuenta.

3.2 Usuario registrado

Usuario que ya tiene una cuenta en el sistema.

Puede:

- Crear, consultar, editar y eliminar gastos.
- Crear, consultar, editar y eliminar ingresos.
- Gestionar sus categorías.
- Ver estadísticas personales.
- Crear presupuestos.
- Participar en grupos de gastos compartidos.

3.3 Administrador

Usuario con permisos especiales para la gestión general del sistema.

Puede:

- Ver usuarios registrados.
- Desactivar cuentas.
- Gestionar categorías globales.
- Revisar errores o incidencias del sistema.

En una primera versión, el rol de administrador puede no implementarse si se quiere simplificar el desarrollo.

4. Requisitos funcionales

RF-01 Registro de usuario

El sistema deberá permitir que un usuario cree una cuenta introduciendo:

- Nombre.
- Correo electrónico.
- Contraseña.

El correo electrónico deberá ser único.

RF-02 Inicio de sesión

El sistema deberá permitir que un usuario inicie sesión mediante su correo electrónico y contraseña.

Si las credenciales son correctas, el sistema devolverá un token de autenticación para usar el resto de servicios protegidos.

RF-03 Cierre de sesión

El sistema deberá permitir que el usuario cierre sesión.

En caso de usar JWT, el cierre de sesión puede gestionarse desde el cliente eliminando el token almacenado.

RF-04 Gestión de gastos

El sistema deberá permitir crear, consultar, editar y eliminar gastos.

Cada gasto deberá tener como mínimo:

- Cantidad.
- Descripción.
- Fecha.
- Categoría.
- Usuario propietario.

Ejemplo:

Un usuario registra un gasto de 25 € con descripción "Cena", fecha "2026-06-28" y categoría "Restaurantes".

RF-05 Gestión de ingresos

El sistema deberá permitir crear, consultar, editar y eliminar ingresos.

Cada ingreso deberá tener como mínimo:

- Cantidad.
- Descripción.
- Fecha.
- Fuente o categoría.
- Usuario propietario.

Ejemplo:

Un usuario registra un ingreso de 800 € con descripción "Sueldo prácticas" y categoría "Trabajo".

RF-06 Consulta de movimientos

El sistema deberá permitir consultar todos los movimientos económicos de un usuario.

Un movimiento puede ser:

- Gasto.

- Ingreso.

El sistema deberá mostrar los movimientos ordenados por fecha, preferiblemente de más reciente a más antiguo.

RF-07 Filtros de búsqueda

El sistema deberá permitir filtrar movimientos por:

- Rango de fechas.
- Tipo de movimiento: gasto o ingreso.
- Categoría.
- Cantidad mínima.
- Cantidad máxima.
- Texto en la descripción.

Ejemplo:

Buscar todos los gastos de “comida” entre el 1 y el 30 de junio.

RF-08 Gestión de categorías

El sistema deberá permitir al usuario crear, consultar, editar y eliminar categorías personalizadas.

Cada categoría deberá tener:

- Nombre.
- Tipo: gasto o ingreso.
- Usuario propietario.

Ejemplos de categorías de gasto:

- Comida.
- Transporte.
- Alquiler.
- Ocio.
- Viajes.
- Salud.

Ejemplos de categorías de ingreso:

- Trabajo.
 - Becas.
 - Transferencias.
 - Ventas.
-

RF-09 Categorías por defecto

El sistema deberá crear categorías por defecto para cada usuario cuando se registre.

Por ejemplo:

- Comida.
 - Transporte.
 - Vivienda.
 - Ocio.
 - Otros.
-

RF-10 Gestión de presupuestos

El sistema deberá permitir al usuario definir presupuestos mensuales.

Un presupuesto deberá tener:

- Mes.
- Año.
- Cantidad máxima.
- Categoría asociada, opcional.
- Usuario propietario.

Ejemplo:

El usuario define un presupuesto de 300 € para comida en julio de 2026.

RF-11 Aviso de presupuesto superado

El sistema deberá poder detectar si un usuario ha superado un presupuesto.

Ejemplo:

Si el usuario tiene un presupuesto de 300 € para comida y ya ha gastado 320 €, el sistema deberá marcar ese presupuesto como superado.

En una versión avanzada, se podría enviar una notificación.

RF-12 Estadísticas básicas

El sistema deberá permitir consultar estadísticas básicas como:

- Total gastado en un mes.
- Total ingresado en un mes.
- Balance mensual.
- Gastos por categoría.

- Evolución de gastos por mes.

Ejemplo:

El usuario consulta cuánto ha gastado en transporte durante junio.

RF-13 Balance económico

El sistema deberá calcular el balance del usuario.

Fórmula:

Balance = Total de ingresos - Total de gastos

El balance podrá calcularse por:

- Día.
 - Mes.
 - Año.
 - Rango personalizado de fechas.
-

RF-14 Gestión de grupos de gastos compartidos

El sistema deberá permitir crear grupos donde varios usuarios puedan registrar gastos compartidos.

Ejemplo:

Un grupo llamado "Viaje a Berlín" con tres usuarios.

RF-15 Añadir miembros a un grupo

El creador de un grupo deberá poder añadir otros usuarios al grupo mediante su correo electrónico.

RF-16 Registrar gasto compartido

Un usuario deberá poder registrar un gasto dentro de un grupo.

Cada gasto compartido deberá tener:

- Cantidad.
- Descripción.
- Fecha.
- Usuario que pagó.
- Usuarios participantes.
- Grupo asociado.

Ejemplo:

Marcos paga 60 € de cena para tres personas. El sistema calcula que cada persona debe 20 €.

RF-17 Cálculo de deudas entre miembros

El sistema deberá calcular cuánto debe cada miembro del grupo y a quién.

Ejemplo:

Si Marcos pagó 60 € y participaron tres personas, los otros dos usuarios deberán 20 € cada uno a Marcos.

RF-18 Marcar deuda como pagada

El sistema deberá permitir marcar una deuda entre usuarios como pagada.

RF-19 Perfil de usuario

El sistema deberá permitir consultar y editar datos básicos del perfil:

- Nombre.
 - Correo electrónico.
 - Contraseña.
-

RF-20 Eliminación de cuenta

El sistema deberá permitir al usuario eliminar su cuenta.

Al eliminar la cuenta, se deberá decidir si:

- Se eliminan todos sus datos.
- Se anonimizan algunos registros compartidos.

Esta decisión dependerá del diseño final del sistema.

5. Requisitos no funcionales

RNF-01 Seguridad

El sistema deberá proteger las rutas privadas mediante autenticación.

Se recomienda usar:

- Spring Security.
- JWT.
- Contraseñas cifradas con BCrypt.

Las contraseñas nunca deberán guardarse en texto plano.

RNF-02 Privacidad

Un usuario solo podrá acceder a sus propios gastos, ingresos, presupuestos y categorías.

No podrá consultar datos de otros usuarios, salvo aquellos compartidos dentro de un grupo.

RNF-03 Rendimiento

El sistema deberá responder a las peticiones habituales en menos de 2 segundos bajo una carga normal.

RNF-04 Escalabilidad

El sistema deberá diseñarse de forma que pueda ampliarse fácilmente para soportar:

- Aplicación móvil.
 - Notificaciones.
 - Sincronización bancaria.
 - Exportación de datos.
 - Más usuarios.
-

RNF-05 Mantenibilidad

El código deberá organizarse en capas:

- Controller.
- Service.
- Repository.
- Model o Entity.
- DTO.
- Security.
- Exception handling.

Esto facilitará el mantenimiento y la ampliación del sistema.

RNF-06 Usabilidad de la API

La API REST deberá usar endpoints claros y coherentes.

Ejemplos:

- GET /api/expenses
 - POST /api/expenses
 - PUT /api/expenses/{id}
 - DELETE /api/expenses/{id}
-

RNF-07 Persistencia

Los datos deberán almacenarse en una base de datos relacional.

Se recomienda usar:

- PostgreSQL para producción.
 - H2 para pruebas.
 - MySQL como alternativa.
-

RNF-08 Fiabilidad

El sistema deberá validar los datos antes de guardarlos.

Por ejemplo:

- No permitir cantidades negativas.
 - No permitir gastos sin fecha.
 - No permitir usuarios sin correo.
 - No permitir presupuestos con cantidad igual o menor que cero.
-

RNF-09 Compatibilidad móvil

La API deberá estar preparada para ser consumida por una aplicación móvil.

Por eso, las respuestas deberán estar en formato JSON.

RNF-10 Documentación

La API debería documentarse usando Swagger/OpenAPI.

Esto permitirá probar los endpoints fácilmente y facilitará el desarrollo de la futura aplicación móvil.

6. Reglas de negocio

RB-01 Cantidades válidas

Un gasto, ingreso o presupuesto deberá tener una cantidad mayor que cero.

RB-02 Propiedad de los datos

Cada gasto, ingreso, categoría y presupuesto deberá pertenecer a un usuario.

Un usuario no podrá modificar información que no le pertenece.

RB-03 Categorías eliminadas

Si se elimina una categoría que ya tiene movimientos asociados, el sistema podrá:

- Impedir la eliminación.
- O mover esos movimientos a una categoría llamada "Sin categoría".

Para una primera versión, se recomienda impedir la eliminación si existen movimientos asociados.

RB-04 Presupuesto mensual único

Un usuario no debería tener dos presupuestos iguales para la misma categoría, mes y año.

Ejemplo:

No debería poder crear dos presupuestos distintos para "Comida" en julio de 2026.

RB-05 Gasto compartido

En un gasto compartido, la suma de las partes individuales deberá coincidir con el total del gasto.

En la versión inicial, se puede dividir el gasto a partes iguales entre todos los participantes.

RB-06 Balance

El balance de un periodo se calculará restando los gastos a los ingresos.

Balance = Ingresos - Gastos

7. Entidades principales del sistema

7.1 User

Representa a un usuario registrado.

Atributos posibles:

- id
- name
- email
- password
- role
- createdAt

Relaciones:

- Un usuario puede tener muchos gastos.
 - Un usuario puede tener muchos ingresos.
 - Un usuario puede tener muchas categorías.
 - Un usuario puede tener muchos presupuestos.
 - Un usuario puede pertenecer a varios grupos.
-

7.2 Expense

Representa un gasto.

Atributos posibles:

- id
- amount
- description
- date
- createdAt
- updatedAt

Relaciones:

- Un gasto pertenece a un usuario.
 - Un gasto pertenece a una categoría.
 - Opcionalmente, un gasto puede pertenecer a un grupo.
-

7.3 Income

Representa un ingreso.

Atributos posibles:

- id
- amount
- description
- date
- createdAt
- updatedAt

Relaciones:

- Un ingreso pertenece a un usuario.
 - Un ingreso pertenece a una categoría.
-

7.4 Category

Representa una categoría de gasto o ingreso.

Atributos posibles:

- id
- name
- type
- createdAt

Relaciones:

- Una categoría pertenece a un usuario.
 - Una categoría puede estar asociada a muchos gastos o ingresos.
-

7.5 Budget

Representa un presupuesto mensual.

Atributos posibles:

- id
- month
- year
- limitAmount
- createdAt

Relaciones:

- Un presupuesto pertenece a un usuario.
 - Un presupuesto puede estar asociado a una categoría.
-

7.6 Group

Representa un grupo de gastos compartidos.

Atributos posibles:

- id
- name
- description
- createdAt

Relaciones:

- Un grupo tiene varios usuarios.
 - Un grupo tiene varios gastos compartidos.
-

7.7 GroupMember

Representa la relación entre usuarios y grupos.

Atributos posibles:

- id
- role
- joinedAt

Relaciones:

- Pertenece a un usuario.
 - Pertenece a un grupo.
-

7.8 SharedExpense

Representa un gasto compartido dentro de un grupo.

Atributos posibles:

- id
- amount
- description
- date
- createdAt

Relaciones:

- Pertenece a un grupo.
 - Tiene un usuario pagador.
 - Tiene varios participantes.
-

7.9 Debt

Representa una deuda calculada entre usuarios.

Atributos posibles:

- id
- amount
- paid
- createdAt
- paidAt

Relaciones:

- Tiene un usuario deudor.
 - Tiene un usuario acreedor.
 - Puede estar asociada a un gasto compartido.
-

8. Casos de uso principales

CU-01 Registrarse

Actor principal: Usuario no registrado.

Flujo principal:

1. El usuario introduce nombre, email y contraseña.
 2. El sistema valida los datos.
 3. El sistema comprueba que el email no exista.
 4. El sistema cifra la contraseña.
 5. El sistema guarda el nuevo usuario.
 6. El sistema crea categorías por defecto.
 7. El sistema devuelve confirmación.
-

CU-02 Iniciar sesión

Actor principal: Usuario registrado.

Flujo principal:

1. El usuario introduce email y contraseña.
 2. El sistema valida las credenciales.
 3. El sistema genera un token JWT.
 4. El sistema devuelve el token al cliente.
-

CU-03 Registrar gasto

Actor principal: Usuario registrado.

Flujo principal:

1. El usuario introduce cantidad, descripción, fecha y categoría.
 2. El sistema valida los datos.
 3. El sistema comprueba que la categoría pertenece al usuario.
 4. El sistema guarda el gasto.
 5. El sistema actualiza los cálculos de presupuesto si corresponde.
 6. El sistema devuelve el gasto creado.
-

CU-04 Consultar gastos

Actor principal: Usuario registrado.

Flujo principal:

1. El usuario solicita sus gastos.
 2. El sistema obtiene los gastos asociados a ese usuario.
 3. El sistema aplica filtros si existen.
 4. El sistema devuelve la lista de gastos.
-

CU-05 Crear presupuesto

Actor principal: Usuario registrado.

Flujo principal:

1. El usuario introduce mes, año, cantidad límite y categoría opcional.
 2. El sistema valida los datos.
 3. El sistema comprueba que no exista ya un presupuesto igual.
 4. El sistema guarda el presupuesto.
 5. El sistema devuelve confirmación.
-

CU-06 Crear grupo de gastos

Actor principal: Usuario registrado.

Flujo principal:

1. El usuario introduce nombre y descripción del grupo.
 2. El sistema crea el grupo.
 3. El sistema añade al usuario como administrador del grupo.
 4. El sistema devuelve el grupo creado.
-

CU-07 Registrar gasto compartido

Actor principal: Usuario miembro de un grupo.

Flujo principal:

1. El usuario introduce cantidad, descripción, fecha y participantes.
 2. El sistema comprueba que el usuario pertenece al grupo.
 3. El sistema valida los participantes.
 4. El sistema guarda el gasto compartido.
 5. El sistema calcula las deudas correspondientes.
 6. El sistema devuelve el resultado.
-

CU-08 Marcar deuda como pagada

Actor principal: Usuario registrado.

Flujo principal:

1. El usuario selecciona una deuda pendiente.
 2. El sistema comprueba que el usuario está implicado en esa deuda.
 3. El sistema marca la deuda como pagada.
 4. El sistema guarda la fecha de pago.
 5. El sistema devuelve confirmación.
-

9. Posibles endpoints REST

Autenticación

Método	Endpoint	Descripción
POST	<code>/api/auth/register</code>	Registrar usuario
POST	<code>/api/auth/login</code>	Iniciar sesión
POST	<code>/api/auth/logout</code>	Cerrar sesión

Usuarios

Método	Endpoint	Descripción
GET	<code>/api/users/me</code>	Consultar perfil
PUT	<code>/api/users/me</code>	Actualizar perfil
DELETE	<code>/api/users/me</code>	Eliminar cuenta

Gastos

Método	Endpoint	Descripción
GET	/api/expenses	Obtener gastos del usuario
GET	/api/expenses/{id}	Obtener un gasto concreto
POST	/api/expenses	Crear gasto
PUT	/api/expenses/{id}	Actualizar gasto
DELETE	/api/expenses/{id}	Eliminar gasto

Ingresos

Método	Endpoint	Descripción
GET	/api/incomes	Obtener ingresos
GET	/api/incomes/{id}	Obtener un ingreso concreto
POST	/api/incomes	Crear ingreso
PUT	/api/incomes/{id}	Actualizar ingreso
DELETE	/api/incomes/{id}	Eliminar ingreso

Categorías

Método	Endpoint	Descripción
GET	/api/categories	Obtener categorías
POST	/api/categories	Crear categoría
PUT	/api/categories/{id}	Actualizar categoría
DELETE	/api/categories/{id}	Eliminar categoría

Presupuestos

Método	Endpoint	Descripción
GET	/api/budgets	Obtener presupuestos
POST	/api/budgets	Crear presupuesto
PUT	/api/budgets/{id}	Actualizar presupuesto
DELETE	/api/budgets/{id}	Eliminar presupuesto

Estadísticas

Método	Endpoint	Descripción
GET	<code>/api/statistics/monthly</code>	Estadísticas mensuales
GET	<code>/api/statistics/categories</code>	Gastos por categoría
GET	<code>/api/statistics/balance</code>	Balance por periodo

Grupos

Método	Endpoint	Descripción
GET	<code>/api/groups</code>	Obtener grupos del usuario
POST	<code>/api/groups</code>	Crear grupo
GET	<code>/api/groups/{id}</code>	Obtener grupo concreto
PUT	<code>/api/groups/{id}</code>	Actualizar grupo
DELETE	<code>/api/groups/{id}</code>	Eliminar grupo

Gastos compartidos

Método	Endpoint	Descripción
GET	<code>/api/groups/{groupId}/expenses</code>	Obtener gastos compartidos
POST	<code>/api/groups/{groupId}/expenses</code>	Crear gasto compartido
GET	<code>/api/groups/{groupId}/debts</code>	Obtener deudas del grupo
PUT	<code>/api/debts/{id}/pay</code>	Marcar deuda como pagada

10. DTOs recomendados

Para no exponer directamente las entidades de la base de datos, se recomienda usar DTOs.

Ejemplos:

RegisterRequest

- name
- email
- password

LoginRequest

- email
- password

AuthResponse

- token
- userId
- name
- email

ExpenseRequest

- amount
- description
- date
- categoryId

ExpenseResponse

- id
- amount
- description
- date
- categoryName

BudgetRequest

- month
- year
- limitAmount
- categoryId

GroupRequest

- name
- description

SharedExpenseRequest

- amount
- description
- date
- payerId
- participantIds

11. Arquitectura recomendada

Se recomienda usar una arquitectura por capas:

Controller

Recibe las peticiones HTTP.

Ejemplo:

- ExpenseController
 - IncomeController
 - CategoryController
 - BudgetController
 - GroupController
-

Service

Contiene la lógica de negocio.

Ejemplo:

- ExpenseService
 - BudgetService
 - GroupService
 - StatisticsService
-

Repository

Accede a la base de datos.

Ejemplo:

- ExpenseRepository
 - UserRepository
 - CategoryRepository
-

Entity

Representa las tablas de la base de datos.

Ejemplo:

- User
 - Expense
 - Income
 - Category
 - Budget
 - Group
 - Debt
-

DTO

Objetos usados para recibir y devolver datos por la API.

Security

Contiene la configuración de autenticación y autorización.

Ejemplo:

- JwtService
 - SecurityConfig
 - UserDetailsServiceImpl
-

12. Modelo UML inicial recomendado

Para el diagrama de clases UML, se recomienda incluir como mínimo las siguientes clases:

- User
- Expense
- Income
- Category
- Budget
- Group
- GroupMember
- SharedExpense
- Debt

Relaciones principales:

- User 1 — * Expense
 - User 1 — * Income
 - User 1 — * Category
 - User 1 — * Budget
 - User * — * Group, mediante GroupMember
 - Group 1 — * SharedExpense
 - SharedExpense 1 — * Debt
 - User 1 — * Debt como deudor
 - User 1 — * Debt como acreedor
-

13. Diagrama de clases orientativo en texto

```
User
- id: Long
- name: String
- email: String
```

- password: String
- role: Role
- createdAt: LocalDateTime

Expense

- id: Long
- amount: BigDecimal
- description: String
- date: LocalDate
- createdAt: LocalDateTime

Income

- id: Long
- amount: BigDecimal
- description: String
- date: LocalDate
- createdAt: LocalDateTime

Category

- id: Long
- name: String
- type: CategoryType

Budget

- id: Long
- month: Integer
- year: Integer
- limitAmount: BigDecimal

Group

- id: Long
- name: String
- description: String
- createdAt: LocalDateTime

GroupMember

- id: Long
- role: GroupRole
- joinedAt: LocalDateTime

SharedExpense

- id: Long
- amount: BigDecimal
- description: String
- date: LocalDate

Debt

- id: Long
- amount: BigDecimal
- paid: Boolean
- paidAt: LocalDateTime

14. Enumeraciones recomendadas

Role

USER,
ADMIN

CategoryType

EXPENSE,
INCOME

GroupRole

OWNER,
MEMBER

15. Validaciones importantes

El sistema deberá validar:

- El email debe tener formato válido.
 - La contraseña debe tener una longitud mínima.
 - La cantidad debe ser mayor que cero.
 - La fecha no debe ser nula.
 - La categoría debe existir.
 - La categoría debe pertenecer al usuario autenticado.
 - El usuario no puede modificar gastos de otro usuario.
 - El presupuesto mensual no debe estar duplicado.
 - Un usuario no puede añadir gastos a un grupo al que no pertenece.
-

16. Tecnologías recomendadas

Backend

- Java 21.
- Spring Boot.
- Spring Web.
- Spring Data JPA.
- Spring Security.
- JWT.
- Bean Validation.
- PostgreSQL.

- Maven o Gradle.

Base de datos

- PostgreSQL para producción.
- H2 para pruebas locales.

Documentación

- Swagger/OpenAPI.

Testing

- JUnit.
 - Mockito.
 - Spring Boot Test.
-

17. Orden recomendado de desarrollo

Fase 1: Base del proyecto

1. Crear proyecto Spring Boot.
 2. Configurar base de datos.
 3. Crear entidades principales.
 4. Crear repositorios.
 5. Probar conexión con base de datos.
-

Fase 2: Usuarios y seguridad

1. Crear registro de usuario.
 2. Crear login.
 3. Configurar JWT.
 4. Proteger endpoints privados.
-

Fase 3: Gastos, ingresos y categorías

1. CRUD de categorías.
 2. CRUD de gastos.
 3. CRUD de ingresos.
 4. Filtros básicos.
-

Fase 4: Presupuestos y estadísticas

1. Crear presupuestos.
2. Calcular gasto mensual.
3. Detectar presupuesto superado.
4. Crear estadísticas básicas.

Fase 5: Grupos y gastos compartidos

1. Crear grupos.
 2. Añadir miembros.
 3. Registrar gastos compartidos.
 4. Calcular deudas.
 5. Marcar deudas como pagadas.
-

Fase 6: Preparación para app móvil

1. Mejorar DTOs.
 2. Documentar API con Swagger.
 3. Añadir paginación.
 4. Añadir filtros avanzados.
 5. Añadir notificaciones en una versión futura.
-

18. Funcionalidades futuras

En futuras versiones se podrían añadir:

- App móvil Android/iOS.
 - Notificaciones push.
 - Escaneo de tickets con OCR.
 - Exportación a Excel o CSV.
 - Conversión entre divisas.
 - Sincronización con cuentas bancarias.
 - Objetivos de ahorro.
 - Gastos recurrentes.
 - Modo offline en la app móvil.
 - Compartir informes.
 - Gráficas avanzadas.
-

19. Conclusión

Este sistema de gestión de gastos estará diseñado como un servicio web escalable, seguro y preparado para ser consumido por una futura aplicación móvil.

La primera versión debería centrarse en usuarios, autenticación, gastos, ingresos, categorías y presupuestos. Una vez que esas funcionalidades estén correctamente implementadas, se podrán añadir estadísticas, grupos compartidos y funcionalidades móviles avanzadas.

Este documento proporciona una base suficiente para comenzar el análisis, realizar diagramas UML y diseñar la arquitectura inicial del proyecto.